

UnboundOS Technical Design and Implementation Specification v2.1.1

Bare-Metal Dataflow Kernel with Local LLM Inference and Fidelity Hardening

Architecture and Core Systems Division

May 2026

Table of Contents

UnboundOS Technical Design and Implementation Specification v2.1.1

Document status: Draft implementation specification

Supersedes: UnboundOS Technical Specification v2.0

Primary target: x86_64 bare-metal dataflow operating environment

Major revision focus: Make the v2.0 design implementable, add a bounded memory model, fix SIMD assumptions, formalize graph verification, add native bare-metal local LLM support, and harden the implementation against convenience creep.

Abstract

UnboundOS is a single-address-space, bare-metal x86_64 operating environment whose primary executable abstraction is a verified dataflow graph rather than a Unix process, thread, or POSIX application. The system is intentionally non-POSIX, non-Unix, and non-sandboxed. It is built for direct hardware ownership, immediate visual graph programming, low-overhead mathematical computation, deterministic diagnostics, and eventually local AI-assisted operation.

This v2.1.1 specification develops the v2.0 prototype concept into an implementation-grade architecture. It replaces the earlier informal assumption of infinite raw RAM with bounded arenas, replaces Boolean wire readiness with epoch-indexed readiness, defines persistent .MOD graphs as symbolic verified artifacts rather than raw pointer dumps, adds a graph-native bare-metal local LLM subsystem, and adds implementation-fidelity constraints that prevent POSIX path leakage, hidden inference loops, direct LLM mutation authority, verifier bypasses, arena leaks, and SIMD capability bypasses. The LLM subsystem is not specified as an Ollama port or a Linux userspace compatibility layer. It is specified as a native tensor-inference runtime exposed through typed UnboundOS graph nodes.

Contents

0. Revision Summary
1. Design Contract
2. Target Profiles and Hardware Assumptions
3. Boot and Platform Contract
4. Memory Management Subsystem
5. Graph and Dataflow Execution Engine
6. Module System and .MOD Graph Format
7. Storage Subsystem
8. Graphics, Input, and Immediate-Mode IDE
9. Exception and Diagnostics Subsystem
10. Bare-Metal Local LLM Subsystem
11. Hardware Acceleration Roadmap
12. Build, Test, and Verification Workflow
13. Implementation Milestones
14. Implementation Fidelity and Conformance Gates
15. Appendices

Conventions

The following requirement terms are used throughout this document:

Term	Meaning
MUST	Required for a conforming implementation.
MUST NOT	Forbidden in a conforming implementation.
SHOULD	Strong recommendation; deviations require a stated reason.
MAY	Optional feature or implementation choice.

Term	Meaning
Stage 0/1/2...	Roadmap maturity level, not a runtime privilege level.

All multi-byte binary formats in this document use little-endian encoding unless explicitly stated otherwise.

0. Revision Summary

0.1 Main Changes from v2.0

The v2.0 document established the core identity of UnboundOS: a bare-metal x86_64 stream-processing unikernel, written in unsafe Rust, with a raw-pointer dataflow engine, .MOD graph persistence, hardware-accelerated math primitives, and SSOD diagnostic halting.

This v2.1.1 revision keeps that identity but makes the design safer to implement, easier to test, and harder to accidentally dilute during implementation.

Major changes:

1. Adds an explicit design contract and trust model.
2. Adds hardware target profiles, including QEMU development, legacy BIOS, modern x86_64, and T500-class legacy hardware.
3. Makes SIMD feature use conditional on CPUID and XCR0 probing.
4. Replaces unbounded bump allocation with named, bounded arenas.
5. Replaces Boolean wire readiness with epoch-indexed readiness.
6. Adds mandatory graph verification before runtime pointer compilation.
7. Expands .MOD from a minimal binary struct into a sectioned symbolic graph format.
8. Adds an implementation-oriented storage roadmap.
9. Adds the missing graphical IDE subsystem.
10. Expands SSOD diagnostics into a complete exception path.
11. Adds the Bare-Metal Local LLM Subsystem.
12. Adds test, verification, and milestone criteria.
13. Adds implementation-fidelity constraints against convenience creep.
14. Defines resource references as opaque IDs rather than POSIX-style paths.
15. Makes graph verification the only legal path to a runtime graph.
16. Adds LLM authority gates, structured-action buffers, and operator approval requirements.
17. Adds quantization, tokenizer, deterministic test, and tensor-kernel conformance details.
18. Adds CI-oriented conformance gates, golden graphs, and parser fuzzing requirements.

0.2 Central Architectural Correction

The core correction in this revision is:

Raw pointers are a private runtime representation, not a persistent graph format and not an external ABI.

Persistent .MOD files MUST store symbolic graph structure: nodes, pins, wire links, type IDs, capabilities, checksums, UI layout, and constant blobs. During load, the kernel verifies the graph and compiles it into raw runtime pointers only after validation succeeds.

0.3 Local LLM Design Position

The local LLM feature is specified as:

A bare-metal tensor-inference subsystem exposed as dataflow graph nodes, with model weights loaded from storage into bounded arenas, executed by CPU/SIMD kernels first, and later accelerated by GPU or device-specific backends.

The system MAY imitate the operator experience of tools such as local model servers, but it MUST NOT require Linux, POSIX, Python, PyTorch, CUDA, Ollama, or llama.cpp at runtime.

0.4 Fidelity-Hardening Position

The most likely failure mode for UnboundOS is not that the design contract is unclear. The most likely failure mode is convenience creep: an implementation shortcut that appears harmless locally but reintroduces a rejected architecture globally.

Examples of forbidden convenience creep:

- treating FAT32 paths as graph-level resource identifiers;
- letting the LLM inference subsystem run as a hidden side loop;
- allowing the IDE to construct runtime graph structures without verifying a symbolic .MOD buffer;
- adding a developer-mode verifier bypass;
- calling an AVX2 tensor kernel without loader-selected dispatch;
- allowing an LLM tool-planning output to mutate a graph without operator approval;
- treating scratch arena exhaustion as a generic panic without arena identity;
- allowing generated code to move directly from text output into execution.

This document therefore specifies not only what UnboundOS does, but which shortcuts a conforming implementation must structurally prevent.

1. Design Contract

1.1 System Identity

UnboundOS is a single-address-space, bare-metal, x86_64 dataflow operating environment. Its primary execution unit is not a process, thread, Unix task, or application package. Its primary execution unit is a verified executable graph.

A graph is compiled into a runtime arrangement of nodes, wires, typed buffers, module entry points, and scheduler metadata. Runtime execution uses raw pointers and trusted Ring 0 code for minimal overhead. Persistent graph files never store live pointers.

UnboundOS deliberately rejects the following as central abstractions:

- POSIX processes;
- Unix signals;
- fork/exec semantics;
- general-purpose shell pipelines as the system core;
- user/kernel isolation as a primary boundary;
- preemptive multitasking as the primary scheduling model;
- dynamically linked ELF application compatibility as an initial target.

1.2 Non-Goals

The first implementation of UnboundOS does not attempt to provide:

- POSIX compatibility;
- Linux binary compatibility;
- Windows binary compatibility;
- memory isolation between modules;
- execution of untrusted code;
- multi-user security;
- package management;
- arbitrary driver support;
- full filesystem compatibility;
- general-purpose web browser support;
- real-time hard guarantees;
- training or fine-tuning of LLMs.

These non-goals prevent architectural drift. UnboundOS is a specialized graph-native bare-metal environment, not a small Unix clone.

1.3 Trust Model

All executable modules run in Ring 0 and are therefore trusted. The system verifies graph structure, memory bounds, type compatibility, declared capabilities, and module availability before execution. It does not sandbox hostile code.

The trust model has three layers:

Layer	Trusted?	Verification target
Kernel core	Yes	Build-time and boot-time invariants
Runtime modules	Yes	Module registration and

Layer	Trusted?	Verification target
		capability declarations
Graph files	No	Structural verification before runtime compilation
Model packages	No	Header, tensor, checksum, and memory verification
Operator input	No	UI confirmation and deterministic validators

A malformed graph or model package **MUST** be rejected. A malicious module is outside the initial threat model.

1.4 Runtime Representation Rule

Persistent files **MUST NOT** contain:

- raw pointers;
- live function addresses;
- machine-specific heap addresses;
- kernel virtual addresses;
- unverified executable code pointers;
- host paths that imply Linux or POSIX semantics;
- dynamically linked library references.

Persistent files **MAY** contain:

- symbolic node type IDs;
- symbolic model IDs;
- section offsets;
- static UI layout metadata;
- typed pin definitions;
- constant blobs;
- checksum tables;
- capability declarations.

The loader is responsible for converting persistent symbolic objects into runtime pointers after verification.

1.5 Failure Doctrine

UnboundOS treats low-level CPU exceptions as diagnostic events. Fatal exceptions **MUST** be captured where possible, rendered to the framebuffer through the SSOD subsystem, emitted to the serial log, and halted deterministically.

Invalid outcomes:

- silent failure;
- uncontrolled reboot loops;
- frozen screen without serial output;
- exception handler recursion without double-fault containment;
- swallowed graph verification errors;
- memory exhaustion without deterministic report.

1.6 Boot-Visible Heartbeat Rule

Every boot path **MUST** produce a serial heartbeat before graphical initialization.

Minimum boot heartbeat:

```
UNBOUNDOS_BOOT_BEGIN
UNBOUNDOS_CPU_PROFILE=<profile>
UNBOUNDOS_MEMMAP_OK=<available_bytes>
UNBOUNDOS_IDT_OK
UNBOUNDOS_BOOT_OK
```

This rule exists so the system can be debugged on QEMU, headless devices, broken framebuffer, and old hardware.

1.7 Operator Authority Rule

The operator is the final authority over graph mutation, storage overwrite, model loading, boot configuration changes, and generated-code execution.

An LLM node **MAY** suggest actions. It **MUST NOT** silently perform system mutations. Every LLM-originated mutation must pass through either:

1. explicit operator approval, or
2. a deterministic verifier whose scope is narrow and auditable.

1.8 No Hidden Execution Rule

All meaningful execution must be represented as graph execution or boot/platform initialization. A conforming implementation **MUST NOT** add hidden task loops, hidden worker threads, hidden queues, or side-process-like subsystems that perform graph-visible work outside the orchestrator.

Permitted exceptions:

- early boot initialization before the orchestrator exists;
- interrupt handlers required for hardware operation;
- diagnostic panic paths;
- future device backends whose command queues are explicitly modeled as graph-visible nodes and capability-gated resources.

The local LLM subsystem is subject to this rule. Tokenization, transformer execution, sampling, detokenization, retrieval, context packing, and assistant actions must be represented as graph nodes or verified macro-nodes.

1.9 Graph Mutation Gate Rule

No code path may mutate a loaded graph directly from an LLM output, parser output, storage read, or debug command. The only legal graph mutation path is:

proposed symbolic graph change

- > temporary UMOD buffer
- > graph verifier
- > operator approval when required
- > runtime graph compilation
- > graph handle swap or reload

The IDE editor itself MUST follow this path. Direct construction of GraphRuntime objects is forbidden outside the graph loader.

1.10 Code Generation Authority Rule

Generated code is text until an operator manually converts it into a module artifact and loads it through the normal module path. UnboundOS MUST NOT provide an eval node, direct generated-code execution node, or LLM-to-module shortcut.

Any future dynamic module loading capability MUST be named explicitly, for example `requires_dynamic_module_load`, and MUST be disabled by default in all initial profiles.

2. Target Profiles and Hardware Assumptions

2.1 Primary Architecture

UnboundOS strictly targets x86_64 for the initial implementation. Other architectures are out of scope for v2.1.1.

The first implementation runs entirely in Ring 0 with a single kernel-owned address space. Paging MAY initially use a flat identity mapping for bring-up, but the permanent memory policy MUST be explicitly documented before real-hardware deployment.

2.2 Hardware Profiles

UnboundOS defines named hardware profiles. A build MAY target one profile while keeping the format and runtime design compatible with others.

Profile	Purpose	Firmware	SIMD baseline	Storage target	Notes
qemu-dev	First	UEFI via	scalar +	IDE/ATA PIO	Fastest

Profile	Purpose	Firmware	SIMD baseline	Storage target	Notes
	implementation and CI target	Limine	optional SSE2/AVX		bring-up path.
legacy-bios	Older real machines	BIOS/VBE via Limine	scalar + SSE2	ATA/SATA compatibility	Avoid AVX assumptions.
modern-x86_64	Current desktop hardware	UEFI GOP	SSE2 + optional AVX2/AVX-512	AHCI/NVMe later	Performance target.
t500-class	Older ThinkPad-class hardware	BIOS/VBE likely	scalar + SSE2	SATA compatibility	Local LLM must use tiny quantized CPU models.

2.3 SIMD Policy

SIMD support is tiered:

1. scalar integer baseline;
2. SSE2 baseline for x86_64 correctness paths;
3. optional AVX;
4. optional AVX2;
5. optional AVX-512;
6. future device-specific acceleration.

A conforming implementation **MUST NOT** assume AVX is present. Modules and model packages **MUST** declare minimum CPU/SIMD requirements. The loader **MUST** reject artifacts whose declared requirements exceed the active hardware profile.

2.4 T500-Class Practicality Note

For T500-class hardware, the practical target is not a high-throughput model server. The practical target is:

- tiny quantized local assistant;
- graph explainer;
- crash-report explainer;
- local documentation search;
- small code or math helper;
- operator-facing IDE assistant.

Modern multi-billion-parameter inference **MAY** be too slow or too memory-heavy on this profile. The architecture still benefits from a tiny model because it exercises tokenizer, model loading, graph integration, streaming output, and local retrieval mechanisms.

3. Boot and Platform Contract

3.1 Bootloader Requirement

The initial bootloader target is Limine. The kernel expects the bootloader to place the CPU in 64-bit long mode before entering the kernel entry point.

Required Limine-provided facilities:

- memory map;
- kernel file location;
- kernel stack or stack handoff contract;
- framebuffer information when available;
- bootloader revision information;
- higher-half direct map information if used by the chosen profile.

3.2 Kernel Entry Contract

The `_start` entry point **MUST** perform only minimal work before establishing serial logging and exception handling.

Required early order:

1. disable interrupts;
2. initialize serial logging;
3. print boot begin heartbeat;
4. validate bootloader handoff structures;
5. install temporary GDT if needed;
6. install early IDT with fatal handlers;
7. ingest memory map;
8. initialize boot allocator;
9. probe CPU features;
10. enable permitted SIMD/FPU state;
11. initialize framebuffer if available;
12. initialize permanent kernel structures;
13. load or embed initial graph;
14. enter orchestrator or IDE shell.

3.3 CPU Feature Detection

The kernel **MUST** use `CPUID` before enabling or using optional CPU features.

Required checks:

- `x86_64` long mode already active;
- SSE/SSE2 availability;
- XSAVE availability before `XCRO` modification;

- AVX availability before AVX use;
- OSXSAVE availability before XGETBV/XSETBV-dependent paths;
- AVX2 and AVX-512 feature bits before dispatching those kernels.

The selected CPU profile MUST be written to serial output.

3.4 FPU and SIMD Initialization

The kernel MUST initialize FPU/SIMD state by feature tier.

Required logic:

1. Clear CR0.EM.
2. Set CR0.MP.
3. Set CR4.OSFXSR for SSE state save/restore support.
4. Set CR4.OSXMMEXCPT for SIMD floating-point exception support.
5. Set CR4.OSXSAVE only when CPUID reports XSAVE support.
6. Use XGETBV/XSETBV only when OSXSAVE is available.
7. Enable only XCR0 bits supported by hardware and required by selected backend.
8. Treat AVX-512/ZMM state separately from AVX/YMM state.

Pseudocode:

```

unsafe fn enable_cpu_math_features(features: CpuFeatureSet) -> SimdTier {
    enable_x87_fpu();

    if features.sse && features.sse2 {
        enable_sse_control_bits();
    } else {
        return SimdTier::Scalar;
    }

    if features.xsave && features.osxsave && features.avx {
        let supported = xgetbv_supported_mask();
        if supported.contains(Xcr0Bits::X87 | Xcr0Bits::SSE | Xcr0Bits::YMM) {
            enable_xcr0(Xcr0Bits::X87 | Xcr0Bits::SSE | Xcr0Bits::YMM);
            return SimdTier::Avx;
        }
    }

    SimdTier::Sse2
}

```

3.5 GDT and IDT Initialization

The kernel **MUST** install an Interrupt Descriptor Table before executing graph code or model kernels. The IDT **MUST** include handlers for at least:

- divide-by-zero;
- invalid opcode;
- general protection fault;
- page fault;
- double fault;
- breakpoint or debug trap in debug builds.

The double-fault handler **SHOULD** use a dedicated Interrupt Stack Table entry once the Task State Segment is initialized.

3.6 Paging Policy

A flat identity map **MAY** be used during early bring-up. Before real-hardware deployment, the active profile **MUST** specify:

- which physical ranges are identity-mapped;
- which framebuffer ranges are mapped;
- whether a higher-half direct map exists;
- whether guard pages are enabled;
- whether null-page access is unmapped;
- whether executable and non-executable page attributes are used.

Even if UnboundOS has no user/kernel isolation, paging is still useful for catching null dereferences, unmapped memory access, and arena guard-zone violations.

3.7 Framebuffer Contract

When a framebuffer is available, the bootloader **MUST** provide:

- base physical address;
- width;
- height;
- pitch;
- bits per pixel;
- color channel layout;
- framebuffer memory size.

The framebuffer driver **MUST** be able to clear the screen, draw monochrome text, and render SSOD diagnostics before the IDE is available.

3.8 Serial Logging Contract

The serial log is the authoritative diagnostic path during bring-up. The framebuffer is optional until graphics initialization succeeds.

Minimum serial functions:

- write byte;
- write string;
- write hexadecimal integer;
- write decimal integer;
- dump CPU feature set;
- dump memory map;
- dump exception frame;
- dump active graph node if available.

3.9 Boot Heartbeat Fallback

If UART probing fails, the kernel **MUST** still record heartbeat events in a reserved boot diagnostic buffer. Once framebuffer output becomes available, the kernel **MUST** print:

`BOOT_NO_SERIAL`

`BOOT_HEARTBEAT_BUFFER_PRESENT`

The phrase “serial heartbeat” therefore means serial output when UART is available and an equivalent deterministic boot-visible output when UART is absent. Boot must never be blind.

4. Memory Management Subsystem

4.1 Memory Philosophy

UnboundOS uses unsafe memory internally, but the memory subsystem **MUST** be bounded, inspectable, and deterministic. The kernel may choose simple allocation strategies, but it **MUST NOT** silently allocate beyond the arena or physical memory range assigned to a subsystem.

The v2.0 phrase “infinite availability of raw RAM” is replaced by explicit arena lifetimes and failure behavior.

4.2 Memory Map Ingestion

At boot, the kernel **MUST** parse the bootloader memory map and classify regions.

Required classes:

- usable RAM;
- reserved firmware memory;
- bootloader reclaimable memory;

- kernel image range;
- framebuffer;
- ACPI or platform tables;
- memory-mapped I/O;
- bad or unavailable memory.

The kernel MUST NOT allocate from reserved, framebuffer, MMIO, or kernel-image ranges.

4.3 Frame Allocator

The first frame allocator MAY be a bitmap or stack allocator. It MUST provide:

- allocate one frame;
- allocate contiguous frames;
- mark range reserved;
- mark range usable;
- dump allocator state to serial;
- fail deterministically on exhaustion.

Frame size is 4096 bytes unless the profile explicitly supports larger pages.

4.4 Arena Model

UnboundOS uses named arenas instead of a single global heap.

Arena	Lifetime	Frees?	Used for
BootArena	boot only	reset after init	boot parsing, temporary tables
KernelArena	whole boot session	no	IDT, GDT, drivers, registries
GraphArena	active graph lifetime	reset on graph unload	nodes, wires, runtime graph
ScratchArena	frame or invocation	frequent reset	temporary buffers
ModelWeightArena	loaded model lifetime	reset on model unload	quantized model weights
InferenceArena	active inference	reset per generation/session	hidden states, logits
KVCacheArena	active chat/session	reset on conversation clear	attention cache
TokenizerArena	loaded model lifetime	reset on model unload	tokenizer tables

Invariant:

No allocator in UnboundOS may silently cross its arena bound. Exhaustion must produce a deterministic loader failure or SSOD fault.

4.5 Arena Ownership and Guard API

Each arena has an owner subsystem and an allowed allocation phase. Implementations SHOULD expose allocation only through guard functions that encode this phase.

Reference guard shape:

```
pub fn with_graph_arena<R>(graph_id: GraphId, f: impl FnOnce(&mut Arena) -> R) -> R;  
pub fn with_model_weight_arena<R>(model_id: ModelId, f: impl FnOnce(&mut Arena) ->  
R) -> R;  
pub fn with_inference_arena<R>(session_id: SessionId, f: impl FnOnce(&mut Arena) ->  
R) -> R;
```

Required ownership rules:

- BootArena may allocate only before permanent kernel initialization completes.
- KernelArena may allocate permanent kernel structures only.
- GraphArena may allocate only during verified graph compilation.
- ScratchArena may allocate only during a graph tick, node invocation, or declared scratch phase.
- ModelWeightArena may allocate only during model load and must become read-only after load when paging support exists.
- InferenceArena may allocate only during an active inference session.
- KVCacheArena may allocate only for a declared model/session pair.

A node that requests an arena outside its declared phase MUST fail with a structured diagnostic.

4.6 Scratch Reset Discipline

Scratch arenas MUST have a deterministic reset point. The default reset point is completion of a top-level graph tick. A graph may also use an explicit ResetArenaNode when that node is declared, verified, and scheduled.

If a node faults before the scratch reset point, the panic or recovery path MUST either reset the scratch arena or mark it poisoned before continuing. Debug builds SHOULD poison scratch memory after reset so stale pointers fault or produce recognizable corruption.

4.7 Read-Only Model Weight Policy

After .UMDL model loading completes, ModelWeightArena SHOULD be marked read-only through paging when the active profile supports it. Write attempts into model weights SHOULD fault and be diagnosed as LLM_MODEL_WEIGHT_WRITE_FAULT or equivalent.

4.8 Aligned Allocation

Alignment is required for SIMD and DMA-oriented future extensions. Each allocation request MUST state an alignment. The allocator MUST reject non-power-of-two alignments.

Aligned address formula:

$$\text{aligned} = (\text{ptr} + (\text{alignment} - 1)) \& \sim(\text{alignment} - 1)$$

Reference allocator contract:

```
pub unsafe fn alloc_aligned(
    arena: *mut Arena,
    size: usize,
    alignment: usize,
) -> Result<*mut u8, AllocError> {
    if !alignment.is_power_of_two() {
        return Err(AllocError::InvalidAlignment);
    }

    let current = (*arena).cursor;
    let aligned = align_up(current, alignment);
    let end = checked_add(aligned, size).ok_or(AllocError::Overflow)?;

    if end > (*arena).limit {
        return Err(AllocError::OutOfArenaMemory);
    }

    (*arena).cursor = end;
    Ok(aligned as *mut u8)
}
```

4.9 Guard Zones and Debug Poisoning

Debug builds SHOULD support arena guard zones. A guard zone is an unmapped or poisoned area placed before or after a critical arena. If paging is available, guard pages SHOULD be unmapped.

Debug builds SHOULD poison reset scratch memory with a recognizable pattern such as 0xCC or 0xA5. Release builds MAY omit poisoning for performance.

4.10 Out-of-Memory Policy

Out-of-memory during load is recoverable. Out-of-memory during execution is usually fatal unless the node explicitly declares recoverable failure semantics.

Context	Behavior
Boot allocator exhausted	SSOD boot fault
Kernel arena exhausted	SSOD kernel fault
Graph load exceeds GraphArena	reject graph load
Model load exceeds ModelWeightArena	reject model load

Context	Behavior
Scratch allocation fails inside node	node fault or SSOD depending on declaration
KV cache full	stop generation or reject longer context

4.11 Memory Diagnostics

Every fatal memory fault MUST report:

- arena name;
- requested size;
- requested alignment;
- arena base;
- arena cursor;
- arena limit;
- active graph ID if any;
- active node ID if any;
- active model ID if any.

5. Graph and Dataflow Execution Engine

5.1 Core Model

The orchestrator replaces the traditional OS process scheduler. It evaluates a verified directed graph of nodes and wires. Each node represents a trusted module function or kernel-provided primitive. Each wire represents a typed buffer and readiness state.

The graph lifecycle is:

persistent .MOD file

- > parse
- > structural verification
- > capability verification
- > memory planning
- > runtime graph compilation
- > orchestrator execution
- > unload/reset

5.2 Node Model

A node has:

- stable node ID;
- node type ID;
- input pin table;
- output pin table;

- capability requirements;
- module entry point after loading;
- internal state pointer after runtime compilation;
- UI coordinates for IDE display;
- diagnostic label.

Runtime node structure:

```
#[repr(C)]
pub struct NodeRuntime {
    pub node_id: NodeId,
    pub type_id: NodeTypeId,
    pub function: ModuleFn,
    pub inputs: *const WireRef,
    pub input_count: usize,
    pub outputs: *const WireRef,
    pub output_count: usize,
    pub internal_state: *mut core::ffi::c_void,
    pub flags: NodeFlags,
}
```

5.3 Wire Model

A wire connects one producer output pin to one or more consumer input pins. Fan-out is legal. Boolean readiness is not sufficient for fan-out because one consumer may clear the flag before another consumer observes it.

Runtime wire structure:

```
#[repr(C)]
pub struct WireRuntime {
    pub wire_id: WireId,
    pub payload_ptr: *mut u8,
    pub size_bytes: usize,
    pub type_id: TypeId,
    pub epoch: u64,
    pub producer_node: NodeId,
    pub consumer_count: u32,
    pub remaining_consumers: u32,
}
```

5.4 Epoch Readiness

Readiness is an epoch-indexed property of producer output relative to downstream consumers.

Rule:

A wire payload is ready for a consumer when the wire epoch is greater than the last epoch observed by that consumer.

Each consumer records the last observed epoch for each input. A producer increments the wire epoch when it writes a new payload. A consumer marks its observation after a successful read. The wire becomes fully consumed when all declared consumers have observed the current epoch.

This fixes fan-out:

```
Producer A -> Wire W -> Consumer B
              |
              +-> Consumer C
```

Consumer B cannot erase W before Consumer C sees it. The scheduler tracks per-consumer observation.

5.5 Pin Typing

Every input and output pin **MUST** declare a type. The graph verifier **MUST** reject wires that connect incompatible pins.

Initial primitive types:

Type ID	Meaning
unit	event pulse with no payload
u8_buffer	raw byte buffer
utf8_buffer	UTF-8 text buffer
token_id_stream	sequence of tokenizer IDs
tensor_q4	quantized 4-bit tensor
tensor_q8	quantized 8-bit tensor
tensor_f32	32-bit float tensor
logits	vocabulary logit vector
sampler_config	sampling configuration
diagnostic_record	structured fault or warning

Type compatibility **MAY** support explicit adapter nodes. Implicit type coercion **SHOULD** be avoided in the first implementation.

5.6 Graph Verification

The graph verifier **MUST** run before any runtime pointer graph is constructed.

Required verification checks:

1. Magic number valid.

2. Version supported.
3. Header length valid.
4. Section table valid.
5. Node count within configured limit.
6. Wire count within configured limit.
7. Every node index resolves.
8. Every wire endpoint resolves.
9. Every pin index exists.
10. Every wire type matches producer and consumer pin declarations.
11. Every node type resolves to a registered module.
12. No undeclared capability is required.
13. No graph cycle exists unless the cycle passes through an explicit delay/state node.
14. All payload sizes are statically known or bounded.
15. Total graph memory fits inside GraphArena.
16. All declared model references resolve or fail gracefully.
17. Checksums match when checksum sections are present.
18. UI layout section does not reference missing nodes.
19. Every constant blob referenced by a node exists in the Constant Blob Section.
20. Every referenced constant blob has the declared byte length and alignment.
21. The scheduling section exists when deterministic mode is requested.
22. Every external reference uses an approved opaque-resource syntax.

A failed check **MUST** produce a structured graph-load error. It **MUST NOT** crash the kernel.

5.7 Single Verifier Gate

The only legal way to create a GraphRuntime is through:

```
pub fn graph_load_from_umod(bytes: &[u8]) -> Result<VerifiedGraph, GraphLoadError>;
pub fn graph_compile_verified(v: VerifiedGraph) -> Result<GraphRuntimeHandle,
GraphCompileError>;
```

A conforming implementation **MUST NOT** provide a developer-mode bypass, quick-load path, editor shortcut, or test-only direct runtime constructor that skips verification. Tests may construct symbolic UMOD buffers more conveniently, but those buffers must still pass through the same verifier.

The verifier **SHOULD** be implemented as one atomic pipeline returning **Result**<VerifiedGraph, GraphLoadError>. Partial runtime graph allocation before verification completes is forbidden except for bounded parser scratch storage.

5.8 Runtime Graph Compilation

After verification, the loader constructs runtime objects in GraphArena.

Compilation steps:

1. allocate node runtime table;
2. allocate wire runtime table;
3. allocate payload buffers with required alignment;
4. resolve node type IDs to function pointers;
5. initialize node state objects;
6. initialize epoch observation tables;
7. attach diagnostics context;
8. publish graph handle to orchestrator.

Raw pointers first appear at this stage.

5.9 Orchestrator Loop

The initial scheduler is cooperative and polling-based. It loops over nodes and fires a node when all inputs are ready according to epoch observation.

Reference shape:

```
pub unsafe fn run_orchestrator(graph: *mut GraphRuntime) -> ! {
  loop {
    for node in graph_nodes(graph) {
      if inputs_ready_for_node(graph, node) {
        set_active_node_for_diagnostics(node.node_id);
        let result = (node.function)(node.inputs, node.outputs, node.internal_state);
        handle_node_result(graph, node, result);
        clear_active_node_for_diagnostics();
      }
    }
  }
}
```

5.10 Cycles and Feedback

Pure combinational cycles are invalid. Feedback is legal only through explicit state or delay nodes.

Examples of legal cycle-breaking nodes:

- DelayOneTick;
- RegisterNode;
- KVCacheNode;
- StateCellNode;
- FrameBufferNode.

The verifier MUST reject unbroken cycles.

5.11 Determinism Rules

A graph is deterministic if:

- node firing order is specified;
- input epochs are observed consistently;
- sampling nodes use explicit seeds or declare nondeterminism;
- time and device input are represented by explicit source nodes;
- storage reads are explicit graph effects.

The first orchestrator SHOULD use stable node order by node ID.

For stronger reproducibility, graph scheduling SHOULD be stored explicitly in a Scheduling Section. A deterministic graph MUST declare one of the following scheduling policies:

Policy	Meaning
node_id_order	Fire ready nodes by ascending node ID.
declared_order	Fire ready nodes in an explicit schedule table.
topological_order	Fire ready nodes in verifier-produced topological order with a stable tie-breaker.

Any source node representing real time, hardware input, random entropy, storage mutation, or device state MUST be tagged as nondeterministic. Its presence disables reproducible-mode guarantees unless the graph provides a deterministic replay source.

6. Module System and .MOD Graph Format

6.1 Purpose

A .MOD file stores a graph. It does not store executable code and does not store live runtime pointers.

The file format is called UMOD at the binary level.

6.2 File Structure

High-level layout:

UMOD Header
Section Table
Node Descriptor Section
Wire Descriptor Section
Pin Type Section
Constant Blob Section
Capability Section
UI Layout Section

External Reference Section
Checksum Section

6.3 UMOD Header

Offset	Size	Type	Field
0x00	4	u32	Magic: "UMOD" = 0x444F4D55
0x04	2	u16	Format major version
0x06	2	u16	Format minor version
0x08	4	u32	Header length
0x0C	4	u32	Section count
0x10	4	u32	Node count
0x14	4	u32	Wire count
0x18	4	u32	Pin type count
0x1C	4	u32	Capability count
0x20	8	u64	Section table offset
0x28	8	u64	File length bytes
0x30	8	u64	Graph stable ID
0x38	8	u64	Header checksum

6.4 Section Descriptor

Offset	Size	Type	Field
0x00	4	u32	Section type
0x04	4	u32	Section flags
0x08	8	u64	Section offset
0x10	8	u64	Section length
0x18	8	u64	Section checksum

6.5 Node Descriptor

Field	Type
node_id	u32
node_type_id	u64
input_pin_base	u32
input_pin_count	u16
output_pin_base	u32
output_pin_count	u16
capability_base	u32
capability_count	u16
ui_x	i32
ui_y	i32
label_offset	u32
constant_ref	u32

6.6 Wire Descriptor

Field	Type
wire_id	u32
src_node_id	u32
src_pin_index	u16
dst_node_id	u32
dst_pin_index	u16
type_id	u64
payload_size	u64
alignment	u32
flags	u32

For fan-out, the persistent format MAY either store multiple wire descriptors that share a source or store a multi-consumer wire section. The verifier must normalize both into the runtime epoch model.

6.7 Capability Declarations

Capabilities are symbolic requirements. Examples:

- requires_framebuffer;
- requires_storage_read;
- requires_storage_write;
- requires_simd_sse2;
- requires_simd_avx2;
- requires_llm_runtime;
- requires_model_weight_arena;
- requires_kv_cache_arena;
- requires_network_future.

The loader MUST reject a graph when any required capability is unavailable or forbidden by the active profile.

6.8 External References

.MOD files MAY reference external artifacts only by opaque resource reference. Resource references are not paths and are not directory walks.

Canonical grammar:

```
resource_ref = resource_type ":" opaque_id
resource_type = "model" | "graph" | "index" | "blob" | "font" | "profile"
opaque_id = 1*64(ALPHA / DIGIT / "_" / "-" / ".")
```

Examples:


```
model:tiny-assistant-q4
index:unboundos-docs-v1
graph:math-primitives-core
blob:boot-font-8x16
```

The legacy-looking scheme `local://models/tiny-assistant.umd` may appear in UI mockups, but the verifier **MUST** normalize or reject it before graph execution. The graph-visible representation **MUST** be an opaque ID, content hash, slot index, or capability token. FAT32 directory parsing is an internal storage-adapter detail and **MUST NOT** leak into graph logic.

Rejected examples:

```
/etc/models/tiny.umd
C:\models\tiny.umd
local://../boot/kernel
models/tiny-assistant.umd
```

6.9 Loader Rejection Rules

The loader **MUST** reject a `.MOD` file if:

- the magic number is invalid;
- the version is unsupported;
- any section overlaps another section illegally;
- any section points outside file bounds;
- checksums fail;
- node or wire counts exceed limits;
- a node type ID is unknown;
- a pin type mismatch exists;
- required memory exceeds GraphArena;
- a required external model is missing;
- required capabilities are unavailable;
- the graph contains an unbroken cycle.

6.10 Binary Format Crate Hygiene

The crate or module defining UMOD binary formats **MUST** use only fixed-width integer types, offsets, lengths, flags, checksums, and symbolic IDs. It **MUST NOT** derive serialization or deserialization for runtime structures containing raw pointers.

Forbidden pattern:

```
#[derive(Serialize, Deserialize)]
pub struct NodeRuntime {
    pub function: ModuleFn,
```

```

    pub inputs: *const WireRef,
}

```

Required pattern:

```

#[repr(C)]
pub struct UmodNodeDescriptor {
    pub node_id: u32,
    pub node_type_id: u64,
    pub input_pin_base: u32,
    pub output_pin_base: u32,
}

```

Integration tests SHOULD scan sample .MOD files for byte sequences resembling canonical kernel virtual addresses and reject any artifact that appears to contain live addresses.

7. Storage Subsystem

7.1 Storage Strategy

Storage is staged. FAT32 compatibility is useful, but it SHOULD NOT be the first storage milestone.

Stage	Storage model	Purpose
0	embedded in-memory .MOD	boot and graph tests without disk
1	raw sector read/write	validate block I/O
2	append-only graph store	simpler persistence than FAT32
3	minimal FAT32	compatibility with host systems
4	AHCI/NVMe	performance and modern hardware

7.2 Resource Namespace Boundary

The storage layer may internally read FAT32 directories, append-only records, raw blocks, or future AHCI/NVMe structures. The graph layer never receives a filesystem path. Graph-level storage references are opaque resource IDs, content hashes, slot IDs, or capability tokens.

Storage read nodes SHOULD accept:

- resource type;
- opaque resource ID;
- expected content type;

- optional checksum;
- capability token if required.

They **MUST NOT** accept arbitrary POSIX-style path strings as graph data.

7.3 ATA PIO Bring-Up

For QEMU and legacy hardware, ATA PIO **MAY** be used as a first disk backend.

Minimum ATA PIO read sequence:

1. select drive/head and LBA high bits;
2. write sector count;
3. write LBA low/mid/high bytes;
4. issue read command 0x20;
5. poll status until data request is ready;
6. read 256 16-bit words from data port 0x1F0;
7. detect error and timeout states.

The storage driver **MUST** include timeouts. Infinite polling is invalid.

7.4 Append-Only Graph Store

Before FAT32, UnboundOS **SHOULD** implement a tiny append-only graph store.

Properties:

- fixed block size;
- append records only;
- record header contains type, length, checksum, and generation;
- latest valid generation wins;
- corrupted tail can be ignored;
- no directory hierarchy required.

This provides reliable graph persistence early without requiring a full filesystem.

7.5 FAT32 Profile

The FAT32 profile **SHOULD** initially support:

- read root directory;
- open file by short name;
- read contiguous or chained cluster file;
- write new file in controlled test mode;
- reject long filename complexity until later;
- never mount untrusted files without bounds checks.

7.6 Storage Diagnostics

Storage errors MUST report:

- device backend;
- LBA or file reference;
- operation type;
- status register if ATA;
- timeout count;
- active graph node if applicable.

8. Graphics, Input, and Immediate-Mode IDE

8.1 Role of the IDE

The UnboundOS IDE is not an application running on an OS. It is part of the operating environment. It is the primary operator interface for constructing, inspecting, running, and debugging graphs.

8.2 Framebuffer Renderer

Minimum framebuffer renderer:

- clear screen;
- draw pixel;
- draw filled rectangle;
- draw line;
- draw monochrome bitmap glyph;
- draw string;
- draw cursor;
- draw scrollable text buffer.

The renderer MUST be callable by SSOD without requiring the full IDE.

8.3 Immediate-Mode UI Primitives

Initial UI primitives:

- panel;
- label;
- button;
- text input;
- scroll view;
- list view;
- node rectangle;

- pin connector;
- wire line;
- inspector field;
- modal confirmation box.

The UI SHOULD be immediate-mode to match the direct state model of UnboundOS.

8.4 Input

Initial input targets:

- PS/2 keyboard in legacy profile;
- USB HID keyboard later;
- PS/2 mouse in legacy profile;
- serial console fallback;
- scripted QEMU input for tests.

The operator MUST be able to reach a useful console or SSOD diagnostic path without mouse support.

8.5 Graph Editor

The graph editor MUST support:

- display nodes;
- display pins;
- display wires;
- select node;
- inspect node metadata;
- create wire;
- delete wire;
- run graph verifier;
- display verification errors;
- save graph;
- load graph;
- preview LLM-suggested graph edits.

The editor MUST save changes to a temporary symbolic UMOD buffer and reload through the verifier. It MUST NOT mutate the live runtime graph directly. Applying edits should be implemented as a verified graph handle swap or full graph reload.

8.6 Inspector Panel

The inspector panel SHOULD expose:

- node ID;
- node type ID;

- pin types;
- capability requirements;
- memory estimate;
- last fired epoch;
- last error;
- linked model ID if applicable;
- debug label.

8.7 Assistant Panel

When local LLM support is present, the IDE MAY expose an assistant panel. The assistant panel MAY explain selected graphs, propose graph patches, summarize SSOD reports, and search local documentation.

The assistant panel MUST NOT mutate graphs silently.

9. Exception and Diagnostics Subsystem

9.1 SSOD Purpose

SSOD means Snark Screen of Death. It is a fatal diagnostic screen that seizes the framebuffer, writes a structured crash report, emits the same report to serial, and halts.

The snark text is optional style. The structured diagnostic fields are mandatory.

9.2 Required Exception Handlers

The kernel MUST install handlers for:

Vector	Name	Mandatory diagnostic fields
0x00	Divide error	RIP, node, operation if known
0x06	Invalid opcode	RIP, opcode bytes if available
0x08	Double fault	RIP, stack pointer, prior fault if known
0x0D	General protection	RIP, error code, active module
0x0E	Page fault	RIP, CR2, error code, active arena

AVX alignment faults may manifest as general protection faults. They are not a separate interrupt vector. The diagnostic layer MAY classify a #GP as probable AVX alignment failure when active module metadata and instruction context support that conclusion.

9.3 Interrupt Stack Frame

Reference structure:

```
#[repr(C)]
pub struct InterruptStackFrame {
    pub instruction_pointer: u64,
    pub code_segment: u64,
    pub cpu_flags: u64,
    pub stack_pointer: u64,
    pub stack_segment: u64,
}
```

Handlers with error codes MUST capture the error code separately.

9.4 Page Fault Handling

The page fault handler MUST read CR2 and report:

- faulting virtual address;
- instruction pointer;
- present bit;
- write/read bit;
- user/supervisor bit;
- reserved-bit violation;
- instruction fetch bit when available;
- active arena if address falls within a known range;
- active graph node if any.

9.5 Panic Path

All unrecoverable kernel errors funnel into:

```
pub fn kernel_panic(reason: PanicReason, context: DiagnosticContext) -> !
```

The panic path MUST:

1. disable interrupts;
2. emit serial report;
3. render framebuffer report if possible;
4. halt CPU in a stable loop.

9.6 Snark Matrix

Fault	Snark output
Divide by zero	“Math called. It said no.”
Invalid opcode	“The CPU looked at your bytes and declined

Fault	Snark output
	the premise.”
Double fault	“You crashed the crash handler. Please restart the universe.”
General protection	“You violated the laws of x86. The borrow checker is laughing.”
Probable AVX alignment	“AVX asked for aligned memory. You delivered modern art.”
Page fault	“You touched memory you do not own. Safety Third is not Physics Optional.”
LLM arena overflow	“The model wanted more RAM than reality provided.”
Graph verification failure	“The graph is not a graph. It is a request for chaos.”

9.7 Structured Diagnostic Record

Every SSOD MUST include:

- kernel version;
- build profile;
- CPU feature profile;
- fault type;
- instruction pointer;
- stack pointer;
- error code if present;
- active graph ID;
- active node ID;
- active model ID if any;
- active arena if any;
- last serial checkpoint;
- recommended next debugging step.

10. Bare-Metal Local LLM Subsystem

10.1 Design Goal

The Bare-Metal Local LLM Subsystem provides local transformer inference inside UnboundOS without requiring Linux, POSIX, Python, PyTorch, CUDA, Ollama, llama.cpp, or any userspace runtime.

The subsystem is not a general machine-learning framework. Its first target is deterministic inference for small-to-medium quantized transformer language models. The implementation

exposes inference through the UnboundOS dataflow graph model: prompts, token streams, logits, sampled tokens, embeddings, generated text, model diagnostics, and retrieval context are graph-level data objects transported through typed wires.

The subsystem **MUST** remain compatible with the core UnboundOS doctrine:

- single address space;
- trusted Ring 0 modules;
- explicit memory arenas;
- verified persistent artifacts;
- deterministic failure paths;
- SSOD diagnostics for fatal violations;
- operator authority over mutations.

10.2 Non-Goals

The initial LLM subsystem does not provide:

- training;
- fine-tuning;
- distributed inference;
- CUDA compatibility;
- PyTorch compatibility;
- Linux driver compatibility;
- arbitrary architecture support;
- GPU acceleration as the first target;
- untrusted model plugins;
- full Ollama compatibility;
- server-style multi-user access.

Host-side tools **MAY** convert external model formats into UnboundOS-native packages. Those converters are development utilities and are not part of the bare-metal runtime.

10.3 Graph-Native Inference Model

A local LLM execution graph is composed from canonical node types.

Node Type	Input	Output	Purpose
TokenizerNode	UTF-8 prompt	token IDs	Converts text to model tokens
EmbeddingNode	token IDs	hidden states	Loads token embeddings
TransformerBlockNode	hidden state, KV cache	hidden state, KV cache	Executes one transformer layer
AttentionNode	Q/K/V tensors	attention output	Performs self-

Node Type	Input	Output	Purpose
FeedForwardNode	hidden state	hidden state	attention Executes MLP projection
NormNode	tensor	tensor	Applies normalization
LogitsNode	final hidden state	logits	Projects to vocabulary scores
SamplerNode	logits, config	next token ID	Applies sampling policy
DetokenizerNode	token IDs	UTF-8 stream	Converts tokens to text
KVCacheNode	generation state	KV buffers	Owns attention cache
ContextPackNode	retrieved chunks	prompt prefix	Builds RAG context

The LLM subsystem **MUST** expose inference as an ordinary UnboundOS graph. Model execution is therefore inspectable, rewritable, partially visualizable, and debuggable inside the UnboundOS IDE.

The implementation **MUST NOT** create a hidden inference thread, hidden model server, hidden side loop, or untracked queue that bypasses graph scheduling. If performance later requires batching, the batcher must itself be a graph-visible node or verified macro-node whose inputs, outputs, memory, and nondeterminism are declared.

10.4 .UMDL Model Package Format

UnboundOS uses .UMDL files for bare-metal language model packages.

.UMDL files contain:

- model architecture metadata;
- tokenizer metadata;
- tensor descriptor table;
- quantization descriptor table;
- weight blob offsets;
- checksum table;
- required CPU/SIMD features;
- required memory estimate;
- maximum context length;
- vocabulary size;
- layer count;
- hidden size;
- attention head count;
- KV-cache layout requirements;

- optional UI-facing model description.

.UMDL files MUST NOT contain:

- live pointers;
- host paths;
- dynamic library references;
- Python code;
- platform-dependent function addresses;
- unverified executable code.

10.5 UMDL Header

Offset	Size	Type	Field
0x00	4	u32	Magic: "UMDL"
0x04	2	u16	Format major version
0x06	2	u16	Format minor version
0x08	4	u32	Header length
0x0C	4	u32	Architecture ID
0x10	4	u32	Quantization scheme ID
0x14	4	u32	Tensor count
0x18	4	u32	Tokenizer section offset
0x1C	8	u64	Tensor descriptor section offset
0x24	8	u64	Weight blob section offset
0x2C	8	u64	Checksum section offset
0x34	8	u64	Required memory bytes
0x3C	8	u64	Required scratch bytes
0x44	8	u64	Required KV-cache bytes per token
0x4C	4	u32	Max context tokens
0x50	4	u32	Vocabulary size
0x54	4	u32	Layer count
0x58	4	u32	Hidden size
0x5C	4	u32	Attention head count

10.6 Quantization Registry

The Quantization scheme ID field is a registry value. The initial implementation SHOULD support a very small subset and reject all unknown schemes.

Initial registry:

ID	Name	Purpose	Required?
0	Q_NONE_F32	Float32 reference tensors	optional
1	Q_NONE_F16	Float16 storage where supported	optional

	ID	Name	Purpose	Required?
	10	Q4_BLOCK32	4-bit block quantization with 32-value blocks	yes for practical small models
	11	Q8_BLOCK32	8-bit block quantization with 32-value blocks	recommended test path
	1000+	implementation-specific	experimental	no

Every quantization scheme MUST define:

- block size;
- scale type;
- zero-point policy;
- byte layout;
- alignment;
- dequantization formula;
- reference test vectors.

Host-side conversion tools MAY import formats such as GGUF and emit .UMDL, but the bare-metal runtime consumes only .UMDL.

10.7 Tokenizer Section

The .UMDL tokenizer section MUST declare its tokenizer type. The first implementation SHOULD support exactly one tokenizer family before expanding.

Initial tokenizer registry:

	ID	Name	Status
	1	BYTE_FALLBACK_BPE	recommended first practical target
	2	SENTENCEPIECE_UNIGRAM	later target
	3	RAW_BYTE_TO_TOKEN	tiny toy-model target

Tokenizer metadata MUST include:

- vocabulary size;
- token table offset and length;
- merge table offset and length when applicable;
- special token IDs: BOS, EOS, PAD, UNK when used;
- UTF-8 validation policy;

- maximum token byte length;
- checksum of tokenizer tables.

A tokenizer round trip test **MUST** exist for every supported tokenizer type.

10.8 Supported Model Architecture Scope

The first model architecture target **SHOULD** be a decoder-only transformer with:

- token embedding table;
- repeated transformer blocks;
- attention with KV cache;
- normalization;
- feed-forward network;
- final vocabulary projection;
- sampling loop.

The implementation **SHOULD** avoid supporting many architectures at once. A single small architecture path is enough for the first successful bare-metal inference milestone.

10.9 LLM Memory Arenas

The LLM subsystem **MUST NOT** allocate from the general kernel arena during token generation. All inference memory is drawn from explicit LLM arenas.

Arena	Purpose	Lifetime
ModelWeightArena	quantized model weights	model loaded
InferenceArena	hidden states, logits, layer buffers	active generation
KVCacheArena	attention key/value cache	active chat/session
TokenizerArena	tokenizer tables and prompt buffers	model loaded
SamplerArena	logits, probabilities, top-k buffers	generation step
ScratchTensorArena	matvec/dequant temporary buffers	one layer or token

A model load request succeeds only if the kernel can reserve all required arenas before execution begins. Partial model load is invalid unless explicitly marked as a streaming-load profile.

Out-of-memory during model load is a recoverable loader failure. Out-of-memory during token generation is a subsystem fault and **MUST** enter deterministic diagnostic handling with active model ID, graph ID, node ID, requested allocation size, and arena state.

10.10 Backend Ladder

The initial LLM backend is CPU-only.

Backend ladder:

1. scalar integer baseline;
2. SSE2 tensor kernels;
3. AVX tensor kernels;
4. AVX2 tensor kernels;
5. AVX-512 tensor kernels;
6. device-specific accelerator backends.

Every model package declares a minimum backend requirement. The loader rejects a model if the active hardware profile cannot satisfy that requirement.

The scalar and SSE2 paths are correctness baselines. AVX, AVX2, and AVX-512 are optimization tiers, not architectural assumptions.

10.11 Required Tensor Primitives

The first implementation generation **MUST** provide the following primitives:

- quantized matrix-vector multiplication;
- dequantization into scratch buffers;
- vector addition;
- vector multiplication;
- RMS normalization or equivalent model-specific normalization;
- rotary position embedding or equivalent positional operation;
- attention score computation;
- softmax;
- top-k selection;
- top-p filtering;
- greedy sampling;
- temperature scaling;
- token embedding lookup;
- final vocabulary projection;
- scaled dot-product attention;
- layer normalization when required by architecture;
- fused dequantize-plus-matvec where a backend provides it;
- NaN/Inf detection in floating-point paths when the active profile enables diagnostic math checks.

Each primitive **MUST** expose:

- input tensor descriptor;

- output tensor descriptor;
- required alignment;
- supported scalar type;
- supported quantization type;
- scratch-memory requirement;
- failure conditions.

10.12 Tensor Descriptor

Reference tensor descriptor:

```
#[repr(C)]
pub struct TensorDesc {
    pub tensor_id: TensorId,
    pub scalar_type: ScalarType,
    pub quant_type: QuantType,
    pub rank: u8,
    pub dims: [u32; 4],
    pub byte_offset: u64,
    pub byte_length: u64,
    pub alignment: u32,
    pub flags: TensorFlags,
}
```

The loader MUST verify that every tensor range falls inside the weight blob or declared runtime arena.

10.13 LLM Reference Test Suite

Every backend tier MUST pass the same reference tests before it is considered conforming.

Required reference tests:

Test	Purpose
Tokenizer round trip	Proves tokenizer tables and special tokens are coherent.
Tiny fixed model logits	Compares known logits for a 1-layer toy transformer.
Deterministic sampling	Same prompt, seed, config, and model produce same tokens.
KV cache append	Incremental generation matches full-prefix recomputation for toy cases.
Quantized matvec vectors	Scalar, SSE2, AVX, and AVX2 paths match reference tolerance.
Softmax stability	Extreme logits do not produce invalid

Test	Purpose
NaN/Inf handling	probability vectors. Invalid numeric state emits diagnostics rather than silent text.

If logits contain NaN or Inf, the node **MUST** emit an LLM diagnostic. The graph may then stop generation, fall back to greedy safe handling, or enter SSOD depending on node policy. Silent propagation is invalid.

10.14 Inference Modes

The LLM subsystem exposes four initial execution modes.

10.14.1 One-Shot Completion

A prompt is submitted and the model generates until an end token, length limit, stop sequence, or fault.

10.14.2 Streaming Completion

Generated tokens are emitted incrementally through a text-output wire. This mode is used by the IDE assistant and serial console assistant.

10.14.3 Embedding Mode

The model produces vector embeddings for local retrieval, search, semantic graph indexing, or document lookup.

10.14.4 Tool-Planning Mode

The model emits structured graph actions rather than free text. This mode is used to assist the IDE in constructing, editing, explaining, or debugging UnboundOS graphs.

Tool-planning output **MUST** be validated before becoming a graph mutation.

10.15 Sampler Configuration

Sampler configuration **MUST** be explicit graph data.

Fields:

- maximum new tokens;
- temperature;
- top-k;
- top-p;
- repetition penalty if implemented;
- stop token list;
- stop string list if detokenizer supports it;
- random seed;

- deterministic mode flag.

If deterministic mode is enabled, sampling **MUST** be reproducible for the same model, prompt, sampler config, and seed.

10.16 IDE Integration

The local LLM may be exposed inside the immediate-mode graphical IDE as an assistant panel.

Initial IDE features:

- explain selected graph;
- suggest missing node connections;
- inspect graph verification failures;
- generate .MOD graph templates;
- explain SSOD crash reports;
- summarize model memory usage;
- propose optimization passes;
- convert natural-language math tasks into dataflow graphs;
- search local documentation;
- provide command suggestions in serial-console mode.

The assistant is advisory. Graph verification remains authoritative.

10.17 Local Retrieval-Augmented Context

UnboundOS **MAY** provide a local retrieval subsystem for documentation, source files, graph libraries, and crash logs.

Retrieval components:

- document chunker;
- local embedding generator;
- vector store;
- lexical index;
- retrieval node;
- context-packing node.

Implementation stages:

Stage	Retrieval feature
1	filename and title search
2	plain text chunk search
3	embedding search
4	hybrid lexical plus vector retrieval

The retrieval subsystem **MUST** operate entirely on local storage. Network access is not required for local assistant operation.

10.18 LLM Authority Model

The local LLM is never a privileged decision-maker.

The LLM may:

- generate text;
- propose graph edits;
- explain diagnostics;
- suggest commands;
- generate code snippets;
- propose model or graph optimizations.

The LLM may not directly:

- write arbitrary memory;
- modify loaded graphs;
- overwrite storage;
- alter boot configuration;
- install modules;
- change hardware state;
- execute generated code;
- suppress diagnostics;
- approve its own graph changes.

Every LLM-originated system action **MUST** pass through operator approval or a deterministic verifier.

10.18.1 Structured Action Buffer

Tool-planning output **MUST** land in a dedicated `structured_action_buffer` wire. The graph mutator **MUST NOT** consume this wire directly.

Required flow:

LLM tool-planning output

- > `structured_action_buffer`
- > action schema validator
- > graph verifier on temporary UMOD patch
- > operator approval UI
- > graph reload or handle swap

The approval UI is an operator-controlled UI element, such as an assistant-panel Apply button. If verification fails, the proposal is discarded and the operator is notified.

10.18.2 Graph Write Lock

Debug and test builds SHOULD provide a `graph_write_lock` flag. While the flag is set, no graph mutation is possible. The flag may be lifted only by operator input or an explicit deterministic test harness. LLM-side code must have no privilege to lift the lock.

10.19 LLM Node in .MOD

Example graph reference:

NodeType: LLMGenerate

ModelRef: model:tiny-assistant-q4

InputPins:

- prompt: utf8_buffer
- sampler_config: sampler_config

OutputPins:

- token_stream: token_id_stream
- text_stream: utf8_stream
- diagnostics: diagnostic_record

Capabilities:

- requires_llm_runtime
- requires_model_weight_arena
- requires_kv_cache_arena

Rule:

.MOD files reference models by stable model IDs or local storage references. They do not embed raw model pointers.

10.20 LLM Diagnostics

LLM subsystem faults MUST report:

- model ID;
- model package checksum;
- architecture ID;
- quantization type;
- active token position;
- max context length;
- KV cache usage;
- arena usage;
- active backend;
- active tensor primitive;
- active graph node.

10.21 LLM Implementation Roadmap

Stage	Goal	Exit criterion
0	Host-assisted mock LLM	IDE graph can send prompt and display returned text
1	Bare-metal tokenizer	Text converts to token IDs and back
2	Tiny fixed model	Hardcoded toy transformer generates deterministic output
3	.UMDL loader	Model header and tensor table verify successfully
4	Quantized CPU inference	Small quantized model generates tokens on CPU
5	SIMD optimization	SSE2/AVX backend dispatch passes correctness tests
6	IDE assistant	Assistant explains graph and SSOD report
7	Local retrieval	Assistant searches local docs and packs context
8	Accelerator research	Experimental device backend behind capability gate

11. Hardware Acceleration Roadmap

11.1 CPU First

The first UnboundOS LLM implementation **MUST** be CPU-first. GPU acceleration is not the first target because bare-metal GPU drivers are complex, hardware-specific, and likely to dominate the project before the graph kernel is stable.

11.2 SIMD Kernel Dispatch

Tensor primitives **SHOULD** use a dispatch table selected at model load.

```
pub struct TensorKernelTable {  
    pub matvec_q4: MatVecFn,  
    pub matvec_q8: MatVecFn,  
    pub rms_norm: UnaryTensorFn,  
    pub softmax: UnaryTensorFn,  
    pub rope: RopeFn,  
    pub top_k: TopKFn,  
}
```

The dispatch table **MUST** be selected from verified CPU features and model requirements.

No tensor primitive should be reachable through an unchecked global call path. Backend-specific symbols such as `matvec_q4_avx2` may exist internally, but graph nodes must reach them only through the loader-selected dispatch table.

Debug builds **SHOULD** assert that the active CPU feature flags match the selected dispatch table before executing a backend-specific primitive.

11.3 DMA and Accelerator Future

Future device acceleration requires:

- physical contiguous buffers or IOMMU-aware mapping;
- DMA-safe arena allocation;
- device reset path;
- command queue abstraction;
- timeout and recovery policy;
- SSOD integration;
- capability gate.

No accelerator backend may bypass graph verification or operator authority.

11.4 External Coprocessor Option

A simpler future path is an external coprocessor interface. UnboundOS could communicate with a dedicated model-running device over a narrow protocol while keeping the main kernel simple.

This profile would still expose LLM behavior as graph nodes, but the tensor kernels would be remote-device backed rather than CPU backed.

12. Build, Test, and Verification Workflow

12.1 Build Requirements

Initial build stack:

- Rust `no_std` kernel crate;
- custom target JSON;
- Limine boot assets;
- QEMU run target;
- serial log capture;
- reproducible disk image build;
- optional host-side `.MOD` and `.UMDL` builders.

12.2 Rust Target Constraints

The kernel crate **MUST** use:

```
#![no_std]
#![no_main]
```

The build **SHOULD** fail if accidental std dependencies are introduced into kernel code.

12.3 QEMU Boot Command

The development build **SHOULD** provide a single command that:

1. builds kernel;
2. creates bootable image;
3. launches QEMU;
4. captures serial output;
5. exits or reports pass/fail for smoke tests.

Example conceptual target:

```
make test-qemu
```

12.4 Minimum Smoke Tests

Test	Pass condition
Boot banner	serial prints UNBOUNDOS_BOOT_OK
Framebuffer	screen clears and prints kernel version
Allocator	aligned allocation returns valid 32- byte boundaries
IDT divide	intentional divide-by-zero enters SSOD
Page fault	bad pointer prints CR2
Graph load	embedded test graph verifies
Scheduler	source -> transform -> sink graph executes once
Fan-out	one producer reaches two consumers
SIMD detection	active SIMD tier matches CPUID result
.MOD reject	malformed graph is rejected, not executed
.UMDL reject	malformed model package is rejected, not loaded

Test	Pass condition
Tokenizer	text -> tokens -> text round trip succeeds
Tiny LLM	fixed toy model emits deterministic token sequence

12.5 Fault Injection Tests

Fault injection MUST include:

- invalid node type ID;
- invalid wire endpoint;
- type mismatch;
- unbroken graph cycle;
- graph memory overrun;
- unsupported SIMD requirement;
- corrupt .UMDL checksum;
- model memory estimate too large;
- KV cache overflow;
- intentional invalid opcode;
- intentional page fault;
- intentional general protection fault when feasible.

12.6 CI Target Matrix

The repository SHOULD define CI-style targets even when running locally.

Target	Required checks
qemu-scalar	boot, allocator, graph verifier, scalar tensor tests
qemu-sse2	SSE2 feature path and tensor tests
qemu-no-avx	proves AVX is not assumed
qemu-avx2-forced-off	proves dispatch obeys active profile, not host CPU
parser-fuzz-umod	malformed .MOD returns structured errors
parser-fuzz-umdl	malformed .UMDL returns structured errors
golden-graphs	required .MOD examples load and run deterministically
golden-models	toy .UMDL models pass reference logits and sampling tests

12.7 Golden Graph Suite

Golden graphs are symbolic .MOD artifacts stored as test fixtures. They **MUST** be loaded through the normal verifier. They **MUST NOT** be constructed directly as runtime structures.

Minimum suite:

- single pulse graph;
- transform graph;
- fan-out graph;
- legal delay cycle graph;
- invalid unbroken cycle graph;
- graph with unknown node type;
- graph with type mismatch;
- LLM streaming toy graph;
- LLM mutation proposal graph that stops at approval.

12.8 Parser Fuzzing Policy

The .MOD and .UMDL parsers are artifact attack surfaces even under a trusted-module model. Fuzzing **MUST** verify that malformed artifacts return structured errors rather than panicking, overflowing, allocating excessive memory, or entering infinite loops.

Fuzzing cases **SHOULD** include:

- overlapping sections;
- offsets outside file bounds;
- undersized headers;
- huge counts with tiny files;
- invalid UTF-8 in labels;
- unknown resource reference syntax;
- unsupported quantization IDs;
- tensor byte ranges outside the weight blob;
- checksum mismatches;
- suspicious pointer-like values in persistent fields.

12.9 Real-Hardware Checklist

Before booting on real hardware:

1. QEMU smoke tests pass.
2. Serial output path verified.
3. SSOD path verified.
4. Disk write path disabled by default or restricted to test image.
5. CPU feature detection logs verified.
6. Framebuffer fallback exists.

7. No infinite polling loops without timeout.
8. Keyboard interrupt handling tested.
9. Storage writes require explicit operator action.
10. Boot media recovery plan exists.

13. Implementation Milestones

13.1 Milestone Overview

Milestone	Name	Result
M0	Boot heartbeat	Kernel boots and logs serial heartbeat
M1	Diagnostics core	IDT and SSOD handle forced faults
M2	Arena memory	Bounded arenas allocate and fail correctly
M3	Embedded graph	Hardcoded graph executes
M4	.MOD loader	Persistent graph verifies and executes
M5	Minimal UI	Framebuffer IDE displays graph
M6	Storage stage 1	Raw sector read works with timeout
M7	Tokenizer	Bare-metal tokenizer runs
M8	Toy transformer	Hardcoded tiny model generates text
M9	.UMDL loader	Model package validates and loads
M10	Quantized inference	Small quantized model streams tokens
M11	IDE assistant	Local assistant explains graph and SSOD
M12	Local retrieval	Assistant searches local docs

13.2 M0 Exit Criteria

- QEMU boots kernel.
- Serial prints boot sequence.
- Kernel reaches halt loop or idle loop intentionally.
- No framebuffer required.

13.3 M1 Exit Criteria

- IDT installed.
- Divide-by-zero handled.
- Page fault handled.
- Invalid opcode handled.
- SSOD serial output includes RIP and reason.

13.4 M2 Exit Criteria

- BootArena, KernelArena, GraphArena, and ScratchArena exist.
- Alignment tests pass.
- Arena exhaustion is deterministic.
- Memory map dump is available.

13.5 M3 Exit Criteria

- Hardcoded graph source -> transform -> sink executes.
- Epoch readiness works.
- Fan-out test passes.
- Active node diagnostics work.

13.6 M4 Exit Criteria

- .MOD parser reads header and section table.
- Verifier rejects malformed graphs.
- Valid graph compiles into runtime graph.
- Runtime graph executes.

13.7 M7-M10 LLM Exit Criteria

The LLM path is acceptable only when:

- tokenizer round trip works;
- .UMDL header verification works;
- tensor descriptors verify;
- model arenas are pre-reserved;
- unsupported SIMD requirements reject correctly;
- tiny model generates deterministic output;
- quantized matvec primitive passes host-generated test vectors;
- streaming token output appears in IDE or serial console.

For T500-class hardware, a practical early target is a tiny toy transformer or a very small quantized model. The success metric is not modern chat throughput. The success metric is a complete local inference path: model package verification, arena reservation, graph execution, deterministic token output, and IDE/serial streaming.

14. Implementation Fidelity and Conformance Gates

14.1 Fidelity Checklist

A conforming implementation SHOULD include the following checks in CI, code review, or build-time linting:

Gate	Required check
No POSIX path leakage	Graph-visible storage refs use opaque IDs, not slash paths.
No hidden inference loop	LLM work is scheduled as graph nodes or verified macro-nodes.
No verifier bypass	GraphRuntime is created only by verified UMOD loading.
No direct LLM mutation	LLM proposals stop at structured_action_buffer and approval UI.
Arena identity always reported	Allocation failures include arena, graph ID, node ID, and requested size.
SIMD dispatch obeys profile	Tensor kernels are selected through checked dispatch tables.
No persistent pointers	UMOD/UMDL formats contain no raw runtime addresses.
Boot never blind	Serial or equivalent deterministic heartbeat is recorded.
No eval node	Generated code remains text until operator-controlled module loading.
Determinism explicit	Reproducible graphs declare schedule, seed, and nondeterministic sources.

14.2 Compile-Time and Review-Time Lints

The project SHOULD eventually enforce the design contract as lints.

Initial lint ideas:

- forbid std in kernel crates;
- forbid / and \ in graph-visible resource reference fixtures;
- forbid serialization derives on runtime pointer structures;
- forbid direct construction of GraphRuntime outside loader module;
- forbid direct calls to backend-specific SIMD tensor symbols outside dispatch-table initialization;
- forbid graph mutation APIs from accepting LLM output types directly;
- require every AllocError::OutOfArenaMemory construction to include arena identity.

14.3 Review Questions

Every implementation change touching storage, graph loading, LLM inference, arenas, SIMD, or IDE mutation should answer:

1. Does this introduce a path around symbolic artifact verification?
2. Does this expose POSIX-like paths above the storage adapter?
3. Does this execute work outside the orchestrator without a graph-visible representation?
4. Does this let the LLM mutate state instead of proposing a change?
5. Does this allocate from an arena outside the declared lifetime phase?
6. Does this assume AVX/AVX2/AVX-512 instead of checking capabilities?
7. Does this make boot, crash, or load failure less diagnosable?
8. Does this weaken deterministic replay guarantees?

14.4 Conformance Language

A prototype may temporarily omit features. It may not silently contradict the design contract. Temporary omissions must be marked as unimplemented. Contradictory shortcuts must be treated as bugs, not as implementation details.

15. Appendices

Appendix A: Minimal Runtime Types

```
pub type NodeId = u32;
```

```
pub type WireId = u32;
```

```
pub type TypeId = u64;
```

```
pub type NodeTypeId = u64;
```

```
pub type TensorId = u32;
```

```
pub type ModuleFn = unsafe fn(
```

```
    inputs: *const WireRef,
```

```
    outputs: *const WireRef,
```

```
    state: *mut core::ffi::c_void,
```

```
) -> ModuleResult;
```

```
#[repr(C)]
```

```
pub struct WireRef {
```

```
    pub wire_id: WireId,
```

```
    pub runtime_ptr: *mut WireRuntime,
```

```
}
```

Appendix B: Fault Code Families

Family	Meaning
BOOT_*	bootloader, CPU, or early init failure
MEM_*	arena, frame, or pointer fault
GRAPH_*	.MOD parse or verification failure
NODE_*	module invocation failure
STORAGE_*	block or filesystem failure
IDE_*	framebuffer or input failure
LLM_*	model, tensor, tokenizer, or inference failure
CPU_*	exception, SIMD, or instruction fault

Appendix C: Required Test Graphs

C.1 Single Pulse Graph

PulseSource -> CounterSink

Purpose: proves basic node firing.

C.2 Transform Graph

ConstBuffer -> UppercaseTransform -> TextSink

Purpose: proves payload transfer.

C.3 Fan-Out Graph

```
ConstBuffer -> SplitWire -> SinkA
              -> SinkB
```

Purpose: proves epoch readiness.

C.4 Delay Cycle Graph

StateCell -> AddOne -> StateCell

Purpose: proves legal feedback through state.

C.5 LLM Streaming Graph

PromptBuffer -> Tokenizer -> LLMGenerate -> Detokenizer -> TextPanel

Purpose: proves graph-native local inference path.

Appendix D: Security and Trust Boundary Statement

UnboundOS is unsafe by design, but not undefined by design. The system intentionally permits trusted Ring 0 module execution and raw pointer runtime graphs. The specification therefore moves safety enforcement to artifact validation, arena bounds, capability checks, deterministic diagnostics, and operator approval boundaries.

The system does not claim to execute hostile modules safely. It does claim that malformed persistent graph and model artifacts should be rejected before execution.

Appendix E: Glossary

Term	Definition
Arena	Bounded memory region with explicit lifetime and failure behavior.
Graph	Executable dataflow structure composed of nodes and wires.
Node	Trusted executable unit with typed input and output pins.
Wire	Typed payload buffer connecting producer and consumer pins.
Epoch	Monotonic version counter for wire readiness.
.MOD	Persistent UnboundOS graph file. Binary magic: UMOD.
.UMDL	Persistent UnboundOS model package file. Binary magic: UMDL.
SSOD	Snark Screen of Death, the fatal diagnostic path.
LLM	Local language model inference subsystem.
Capability	Symbolic requirement declared by a graph, node, or model package.
Backend	CPU/SIMD/device implementation of tensor primitives.
Operator	Human controlling the system.

Appendix F: Core Doctrine Summary

The shortest complete description of v2.1.1 is:

UnboundOS is a graph-native bare-metal operating environment. It runs trusted Ring 0 modules in a single address space, but persistent artifacts are symbolic and verified before becoming raw runtime pointers. Memory is explicit and arena-bounded. Diagnostics are deterministic. Local LLM support is implemented as native tensor inference inside the graph engine, not as a port of a Linux userspace stack.

Appendix G: ASCII Architecture Diagrams

G.1 Graph Lifecycle

UMOD bytes

|
v

parse header and section table
|
v
verify structure, types, capabilities, memory, external refs
|
v
VerifiedGraph
|
v
compile into GraphArena runtime tables
|
v
GraphRuntimeHandle
|
v
orchestrator execution
|
v
unload, reset arenas, release handle

G.2 LLM Inference Graph

PromptBuffer
|
v
TokenizerNode ---> token_id_stream
|
v
EmbeddingNode
|
v
TransformerBlockNode[0]
|
v
TransformerBlockNode[1..N]
|
v
NormNode
|
v
LogitsNode
|
v
SamplerNode -- next token --> KVCacheNode

|
v
DetokenizerNode
|
v
TextPanel or SerialConsole

G.3 Safe LLM Graph Patch Flow

LLM text or tool-planning output

|
v
structured_action_buffer
|
v
action schema validator
|
v
temporary UMOD patch
|
v
graph verifier
|
v
operator approval UI
|
v
verified graph reload or handle swap

G.4 Arena Lifetimes During Local Inference

model load:

TokenizerArena allocate tables

ModelWeightArena allocate weights, then mark read-only when possible

session start:

KVCacheArena reserve context cache

InferenceArena reserve hidden/logit buffers

per token:

ScratchTensorArena reset each token or layer phase

SamplerArena reset each token

session end:

InferenceArena reset

KVCacheArena reset or retained by explicit session policy

model unload:

```
TokenizerArena  reset
ModelWeightArena reset
```

Appendix H: Minimal UMOD Example

This appendix gives a normative shape for a minimal valid two-node graph. The byte offsets are illustrative; exact checksums are placeholder values for documentation.

Graph:

ConstTextNode -> TextSinkNode

Semantic description:

Object	Value
Node 1	ConstTextNode, output pin 0: utf8_buffer
Node 2	TextSinkNode, input pin 0: utf8_buffer
Wire 1	Node 1 output 0 to Node 2 input 0
Constant blob	ASCII hello
Capability	framebuffer or serial text sink, depending on profile

Illustrative section table:

UMOD Header

```
magic = UMOD
major = 1
minor = 0
node_count = 2
wire_count = 1
```

Section[0] = Node Descriptor Section

Section[1] = Wire Descriptor Section

Section[2] = Pin Type Section

Section[3] = Constant Blob Section

Section[4] = Capability Section

Section[5] = Checksum Section

The verifier must prove that both node type IDs resolve, the wire connects compatible pins, the constant blob exists, the payload size matches the declared utf8_buffer length, and all required capabilities are available.

Appendix I: Glossary and Acronym Table

Term	Meaning	First relevant section
ABI	Application Binary Interface; here, the call and layout contract for modules	5, 6
Arena	Bounded memory region with explicit lifetime and failure behavior	4
Capability	Symbolic requirement declared by a graph, node, model, or backend	1, 6
CPUID	x86 instruction used to discover CPU feature support	3
GraphRuntime	Runtime pointer-based graph representation produced after verification	5
KV cache	Key/value attention cache used during autoregressive transformer inference	10
LLM	Local language model inference subsystem	10
Macro-node	Verified node that internally represents a larger graph fragment or batched operation	1, 10
ModelWeightArena	Arena holding loaded model weights	4, 10
Opaque resource ID	Non-path identifier used by graphs to reference storage objects	6, 7
RoPE	Rotary positional embedding	10
SSOD	Snark Screen of Death; fatal structured diagnostic path	9
UMOD	Binary graph file format backing .MOD	6
UMDL	Binary model package file format backing .UMDL	10
XCRO	x86 extended control register governing enabled extended processor states	3

Appendix J: Fidelity Threat Matrix

Threat	Failure mode	Required countermeasure
Path leakage	Graphs depend on Unix-like file paths	Opaque resource references only
Inference side loop	LLM runs outside orchestrator	Graph nodes or verified macro-nodes only
Silent LLM mutation	Assistant edits graph without approval	Structured buffer plus verifier plus operator gate
Arena leak	Scratch or inference memory grows invisibly	Reset discipline, poisoning, diagnostic identity
Verifier bypass	Runtime graph built directly	Loader is sole constructor
Persistent pointer leakage	Host builder serializes addresses	Fixed-width descriptors only; address scan tests
SIMD assumption	AVX path called on unsupported CPU	Loader-selected dispatch table only
Blind boot	No serial or fallback output	UART probe plus boot diagnostic buffer
Eval creep	Generated code executes directly	No eval node; module loading capability gated
Nondeterministic drift	Firing order or entropy source untracked	Scheduling section and nondeterministic-source tags